

Sorting

Sorting refers to arranging data in a particular format. Sorting algorithm specifies the way to arrange data in a particular order. Most common orders are in numerical or lexicographical order.

Sorting algorithms are evaluated based on several important properties that determine their suitability for different types of problems.

Stability refers to whether the algorithm preserves the relative order of elements having equal keys; this is crucial when sorting records with multiple fields.

In-place sorting indicates that the algorithm requires only a constant amount of extra memory apart from the input array, making it suitable for memory-constrained environments.

space complexity measures the additional memory required during execution.

Time complexity of a sorting algorithm is analyzed in terms of best, average, and worst cases, which helps in understanding its performance under different input conditions

Bubble Sort

Bubble Sort (also called Exchange Sort) is an algorithm that repeatedly compares adjacent elements and exchanges them if they are in the wrong order. Each pass places the largest element at the end of the list.

Bubble Sort Algorithm

Algorithm (Bubble Sort) Given a linear array A with N elements, rearrange the elements of A in ascending order.

Algorithm bubbleSort(A, N)

1. for $l \leftarrow 1$ to $N - 1$ do
 - a. for $J \leftarrow 1$ to $N - l$ do
 - i. if $A[J] > A[J + 1]$ then interchange $A[J]$ and $A[J + 1]$
 - ii. end if
 - b. end for

UNIT-2

2. end for
3. return A

Example:

Consider the Array [4, 2, 7, 1].

First pass:

- Compare 4 and 2, swap to get [2, 4, 7, 1]
- Compare 4 and 7, no swap
- Compare 7 and 1, swap to get [2, 4, 1, 7]

Second pass:

- Compare 2 and 4, no swap
- Compare 4 and 1, swap to get [2, 1, 4, 7]

Third pass:

- Compare 2 and 1, swap to get [1, 2, 4, 7]

Fourth pass:

- List is already sorted, no swaps needed

Final sorted list: [1, 2, 4, 7].

Bubble Sort – Time Complexity Analysis

Case	Complexity	Reason
Best Case	$O(n)$	Occurs when the list is already sorted. Bubble Sort makes one full pass and performs only comparisons; no swaps take place. (If the optimized version with a swap-flag is used, the algorithm terminates after the first pass.)
Worst Case	$O(n^2)$	Occurs when the list is in reverse order. Each element must bubble through all remaining elements, requiring maximum comparisons and swaps across all passes.
Average Case	$O(n^2)$	For a randomly ordered list, Bubble Sort performs approximately $n(n-1)/2$ comparisons, leading to quadratic behavior.

UNIT-2

Implementation in C++

```
#include <iostream>
#include <vector>
using namespace std;

// Bubble Sort Algorithm
void bubbleSort(vector<int> &A)
{
    int N = A.size();

    // [Initialize passes.]
    for (int I = 0; I < N - 1; I++)
    {
        // [Scan unsorted portion.]
        for (int J = 0; J < N - I - 1; J++)
        {
            // [Compare adjacent elements.]
            if (A[J] > A[J + 1])
            {
                // [Swap.]
                int TEMP = A[J];
                A[J] = A[J + 1];
                A[J + 1] = TEMP;
            }
        }
    }
}

// ----- Driver Code -----
int main()
```

UNIT-2

```
{
    int N;
    cout << "Enter number of elements: ";
    cin >> N;

    vector<int> A(N);

    cout << "Enter " << N << " elements:\n";
    for (int i = 0; i < N; i++)
    {
        cin >> A[i];
    }

    bubbleSort(A);

    cout << "\nSorted array:\n";
    for (int x : A)
    {
        cout << x << " ";
    }
    cout << endl;

    return 0;
}
```

Insertion Sort

Insertion Sort is a simple comparison-based sorting algorithm that builds the final sorted list one element at a time. It works similarly to the way you might sort playing cards in your hands.

The algorithm takes one element from the unsorted portion of the list and inserts it into its correct position in the sorted portion. This process is repeated until the entire list is sorted.

Insertion Sort Algorithm

Algorithm (Insertion Sort) Given a linear array A with N elements, sort A in ascending order by inserting each element into its proper position.

Algorithm insertionSort(A, N)

1. set $A[0] \leftarrow -\infty$
2. for $K \leftarrow 2$ to N do
 - a. set $TEMP \leftarrow A[K]$
 - b. set $PTR \leftarrow K - 1$
 - c. while $TEMP < A[PTR]$ do
 - i. set $A[PTR + 1] \leftarrow A[PTR]$
 - ii. set $PTR \leftarrow PTR - 1$
 - d. end while
 - e. set $A[PTR + 1] \leftarrow TEMP$
3. end for
4. return A

Example:

Consider the list [4, 2, 7, 1].

- First pass ($i = 1$):
 - Key is 2. Compare 2 with 4.
 - 2 is less than 4, so move 4 to the right and insert 2 at the beginning to get [2, 4, 7, 1].

Second pass ($i = 2$):

- Key is 7. Compare 7 with 4.
- 7 is greater than 4, so no movement is needed. List remains [2, 4, 7, 1].

Third pass ($i = 3$):

- Key is 1. Compare 1 with 7, 4, and 2.
- 1 is less than 7, move 7 to the right.
- 1 is less than 4, move 4 to the right.
- 1 is less than 2, move 2 to the right.
- Insert 1 at the beginning to get [1, 2, 4, 7].

Final sorted list: [1, 2, 4, 7].

Insertion Sort – Time Complexity Analysis

Case	Complexity	Reason
Best Case	$O(n)$	Occurs when the array is already sorted. Each element is compared once with the previous element; no shifting is required.
Worst Case	$O(n^2)$	Occurs when the array is in reverse order. Each insertion shifts all earlier elements one position to the right, giving $1 + 2 + \dots + (n-1)$ shifts.
Average Case	$O(n^2)$	For random input, on average each inserted element is compared with half of the sorted part, giving roughly $n^2/4$ operations.

Implementation in C++

```
#include <iostream>
#include <vector>
using namespace std;

// Insertion Sort Algorithm
void insertionSort(vector<int> &A)
{
    int N = A.size();

    // [Process each element.]
    for (int K = 1; K < N; K++)
    {
        // [Store current element.]
        int TEMP = A[K];
        int PTR = K - 1;

        // [Shift larger elements.]
        while (PTR >= 0 && A[PTR] > TEMP)
        {
            A[PTR + 1] = A[PTR]; // move element forward
            PTR--;              // move backward
        }

        // [Insert element.]
```

UNIT-2

```
    A[PTR + 1] = TEMP;
}
}

// ----- Driver Code -----
int main()
{
    int N;
    cout << "Enter number of elements: ";
    cin >> N;

    vector<int> A(N);

    cout << "Enter " << N << " elements:\n";
    for (int i = 0; i < N; i++)
    {
        cin >> A[i];
    }

    insertionSort(A);

    cout << "\nSorted array:\n";
    for (int x : A)
    {
        cout << x << " ";
    }
    cout << endl;

    return 0;
}
```

Selection Sort

Selection Sort is a simple comparison-based sorting algorithm. It divides the list into two parts: the sorted part at the beginning and the unsorted part at the end.

UNIT-2

The algorithm repeatedly selects the smallest (or largest, depending on sorting order) element from the unsorted part and swaps it with the first element of the unsorted part, effectively growing the sorted part by one element with each iteration.

Selection Sort Algorithm

Algorithm (Selection Sort) Given a linear array A with N elements, sort A in ascending order by repeatedly selecting the smallest element from the unsorted portion.

Algorithm selectionSort(A, N)

1. *for $K \leftarrow 1$ to $N - 1$ do*
 - a. *Call $\text{MIN}(A, K, N, \text{LOC})$, where LOC is the position of the smallest element in the subarray $A[K \dots N]$.*
 - b. *set $\text{TEMP} \leftarrow A[K]$*
 - c. *set $A[K] \leftarrow A[\text{LOC}]$*
 - d. *set $A[\text{LOC}] \leftarrow \text{TEMP}$*
2. *end for*
3. *return A*

Example:

Consider the list $[4, 2, 7, 1]$.

• First pass:

- Find the minimum element in $[4, 2, 7, 1]$, which is 1
- Swap 1 with 4 to get $[1, 2, 7, 4]$

Second pass:

- Find the minimum element in $[2, 7, 4]$, which is 2
- Swap 2 with 2 to get $[1, 2, 7, 4]$ (no change)

Third pass:

- Find the minimum element in $[7, 4]$, which is 4
- Swap 4 with 7 to get $[1, 2, 4, 7]$

Fourth pass:

UNIT-2

- The last element 7 is already in place, no need to swap

Final sorted list: [1, 2, 4, 7].

Selection Sort – Time Complexity Analysis

Case	Complexity	Reason
Best Case	$O(n^2)$	Even if the array is already sorted, Selection Sort still scans the remaining unsorted portion ($A[K..N]$) in each pass to find the minimum. No comparisons are saved.
Worst Case	$O(n^2)$	In every pass, minimum search scans all remaining elements; comparisons remain the same regardless of arrangement.
Average Case	$O(n^2)$	For random input, the number of comparisons does not change; the algorithm always performs about $n^2/2$ comparisons.

Implementation in C++

```
#include <iostream>
#include <vector>
using namespace std;

// Selection Sort Algorithm
void selectionSort(vector<int> &A)
{
    int N = A.size();

    // [Initialize passes.]
    for (int K = 0; K < N - 1; K++)
    {
        // [Find smallest element.]
        int LOC = K;
        for (int i = K + 1; i < N; i++)
        {
            if (A[i] < A[LOC])
            {
                LOC = i;
            }
        }
    }
}
```

UNIT-2

```
// [Swap into place.]
int TEMP = A[K];
A[K] = A[LOC];
A[LOC] = TEMP;
}
}

// ----- Driver Code -----
int main()
{
    int N;
    cout << "Enter number of elements: ";
    cin >> N;

    vector<int> A(N);

    cout << "Enter " << N << " elements:\n";
    for (int i = 0; i < N; i++)
    {
        cin >> A[i];
    }

    selectionSort(A);

    cout << "\nSorted array:\n";
    for (int x : A)
    {
        cout << x << " ";
    }
    cout << endl;

    return 0;
}

= C[3] = 7
```

UNIT-2

C = [2, 2, 4, 6, 7, 8] Updated C

(d)

B = [_, 0, _, _, _, _, 3, _] Place A[7] = 0, Position = C[0] = 2

C = [1, 2, 4, 6, 7, 8] Updated C

(e)

B = [_, 0, _, _, _, 3, 3, _] Place A[6] = 3, Position = C[3] = 6

C = [1, 2, 4, 5, 7, 8] Updated C

(f)

B = [0, 0, 2, 2, 3, 3, 3, 5] Sorted output array B

Time Complexity Analysis

Case	Complexity	Reason
Best Case	$O(n + k)$	All loops always execute fully regardless of input order. Initialization (k), counting (n), cumulative sum (k), and output placement (n) all run.
Worst Case	$O(n + k)$	Counting Sort performs the same number of operations for every input; values and ordering do not affect the loop sizes.
Average Case	$O(n + k)$	Deterministic algorithm; expected operations equal worst case.

Where:

- n = number of elements in A
- k = range of key values (0 to k)

C++ Implementation

```
#include <iostream>
#include <vector>
using namespace std;

// Counting Sort Algorithm
vector<int> countingSort(const vector<int> &A, int k)
{
    int n = A.size();

    // [Initialize counts.]
    vector<int> C(k + 1, 0);
```

UNIT-2

```
vector<int> B(n);

// [Count occurrences.]
for (int j = 0; j < n; j++)
{
    C[A[j]] = C[A[j]] + 1;
}

// [Compute cumulative counts.]
for (int i = 1; i <= k; i++)
{
    C[i] = C[i] + C[i - 1];
}

// [Build output array.]
for (int j = n - 1; j >= 0; j--)
{
    B[C[A[j]] - 1] = A[j]; // place element
    C[A[j]] = C[A[j]] - 1; // update position
}

// [Return.]
return B;
}

// ----- Driver Code -----
int main()
{
    int n, k;

    cout << "Enter number of elements: ";
    cin >> n;

    vector<int> A(n);

    cout << "Enter elements (range 0 to k):\n";
```

UNIT-2

```
for (int i = 0; i < n; i++)
{
    cin >> A[i];
}

cout << "Enter maximum value k: ";
cin >> k;

vector<int> B = countingSort(A, k);

cout << "\nSorted array:\n";
for (int x : B)
{
    cout << x << " ";
}
cout << endl;

return 0;
}
```